

SGLBP: Subgraph-based Local Binary Patterns for Feature Extraction on Point Clouds

Bao Guo, Yuhe Zhang, Jian Gao, Chunhui Li and Yao Hu

School of Information Science and Technology, Northwest University, Xuefu Road, No. 1, Changan District, 710127, Xi'an, China

Abstract

Extraction for points that can outline the shape of a point cloud is an important task for point cloud processing in various applications. The topology information of the neighbourhood of a point usually contains sufficient information for detecting features, which is fully considered in this study. Therefore, a novel method for extracting feature points based on the topology information is proposed. First, an improved α -shape technique is introduced, generating two graphs for potential feature detection and neighbourhood description, respectively. Local binary pattern (LBP) is then applied to the subgraphs, thus subgraph-based local binary patterns (SGLBPs) are generated for encoding the topology of the neighbourhoods of points, which helps to remove non-feature points from potential feature points. The proposed method can directly process raw point clouds and needs no prior surface reconstruction or geometric invariants computation; furthermore, the proposed method detects feature points by analysing the topologies of the neighbourhoods of points, consequently promoting the effectiveness for tiny features and the robustness to noises and non-uniformly sampling patterns. The experimental results demonstrate that the proposed method is robust and achieves state-of-the-art performance.

Keywords: CAD, modelling, point-based graphics

CCS Concepts: • Computing methodologies → Modelling and simulation

1. Introduction

With the rapid development of scanning devices, the use of point clouds in various fields [XZW*15, DWXW21], such as 3D animation [NS20], reverse engineering [GB21], object reconstruction [NLNZ16, NAK*18], pattern recognition [SCZ21] and computer-aided design [DHL*20], has drawn considerable attention. Thus, as the fundamental processing task for point clouds, automatically and directly extracting feature points from raw point clouds without other pretreatment steps is a significant and challenging problem.

The foci of feature extraction for point clouds vary depending on the application, and features can refer to the feature points [GZZ21], feature lines [JYH*13, DVC18], feature point pairs [GZWW19] or semantic features [VTHLB15]. Some features indicate the points that can entirely outline the shape of the models [YFFS20, Zho09], whereas in registration or matching for point clouds, features usually refer to the points that are considered corresponding point pairs [RBB09]. However, feature lines can also outline the shape of the models, which can be constructed from the feature points [SHWW20] or directly extracted from the

point clouds [YLT*20]. In the current study, we focus on extracting the feature points that entirely outline the shapes of the models in which both sharp and smooth features are considered important.

Previous feature extraction methods that detect feature points for describing the shape of the models can be roughly classified into non-learning-based methods and deep-learning-based methods. Non-learning-based methods typically implement various strategies on the point coordinates or geometric invariants [PKG03, DVVR07], rely on surface segmentation or fitting [DIOHS08, BL08] or take the reconstructed meshes or voxelization models as input [SF04, GY19]. Although some methods depend on the meshes that contain underlying connection information between points, they still need to compute the geometric invariants, such as Gao and Yamaguchi [GY19]. For learning-based methods, such as EC-Net [JTYX18] and PIENet [WXX*20] have been designed to directly detect edges and boundaries on point clouds and have achieved satisfactory results. However, both approaches exhibit sensitivity to non-uniform sampling and insufficient robustness to extract tiny features. Machine or deep-learning-based methods also

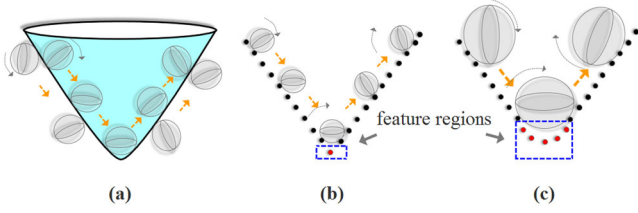


Figure 1: The illustration of the two rolling balls. (a) Two balls roll on the surface; (b) a small ball capture sharp features; (c) a large ball capture smooth features.

have many disadvantages, such as high hardware costs and massive training data.

Observation of point clouds indicates that sharp features usually appear around the boundaries of curved surfaces with sharp angles between them. Thus we consider two spheres with an adaptively computed radius as balls rolling on both the inner and outer surfaces of point clouds. The boundaries of curved surfaces will prevent one or both of the rolling balls from reaching the feature points, as shown in Figure 1(a). Therefore, the points that cannot be ‘reached’ by the balls are considered points in feature regions, as shown in Figures 1(b) and (c). However, the computation of an appropriate scale of the ball that can accurately detect the feature points is non-trivial since the point cloud is usually non-uniformly sampled and contains both sharp and smooth features (as shown in Figures 1(b) and (c)). Furthermore, whether a point is a feature point usually depends on its local neighbouring information, rendering the description and analysis of the neighbourhood also significant for feature recognition. Therefore, an improved α -shape is used for both potential feature extraction and neighbourhood description. Specifically, the subgraph of the created α -shape graph containing the neighbouring points of a point is regarded as the topology of the neighbourhood in our study.

Given the potential feature points and their graph-based neighbourhood, we then introduce subgraph-based local binary patterns (SGLBPs), which applies local binary patterns (LBPs) [OPH96] to the subgraphs representing the topologies of neighbourhoods (created in the previous step). Six useful patterns are thereby defined for analysing SGLBPs, thus iteratively recognizing and removing the non-feature points from the potential feature points, and the remaining points are identified as feature points.

The proposed method aims at directly processing raw point clouds by only calculating the Euclidean distances between points, which needs no prior surface reconstruction, mesh generation, re-sampling operation or any geometric invariants computation. Furthermore, the feature extraction is completed by analysing the topologies of the neighbourhoods instead of any potential processings, such as potential smooth surface recognition, curve fitting or part segmentation, consequently promoting the sensitivity to tiny features and the robustness to noises and non-uniformly sampling.

Our main contributions are as follows:

1. An improved α -shape technique is presented for both potential feature extraction and neighbourhood description.

2. SGLBP codes with several useful patterns are ultimately introduced to iteratively extract features.
3. For evaluating feature extraction results, iterative closest point (ICP) is introduced to determine the point pairs that are used for computing the metrics; furthermore, a novel metric is introduced to evaluate how many tested point clouds are successfully processed.

2. Related Works

Previous research on feature detection in 3D point clouds, as well as α -shapes and LBP, are summarized below.

Non-learning-based methods. Early studies on feature extraction can be roughly classified into three main categories: methods based on the analysis on point coordinates or geometric invariants, methods based on potential surface segmentation or fitting and methods based on processing voxels or meshes that are reconstructed from the original point clouds.

There are several methods that can be classified into the first category. Gumhold *et al.* [GWM01] used covariance analysis to compute a probability of a point belonging to different kind of features. Geometric invariant-based methods all require to calculate the geometric invariant information [DM14, LCBM21] and their outputs significantly rely on the accuracy of the invariant information. For example, Pauly *et al.* [PKG03] performed a multi-scale analysis of the curvatures of points. Weber *et al.* [WHH10] applied clustering of the normal vectors of the neighbouring points for sharp feature detection. By improving the method in Weber *et al.* [WHH10], Bazezian *et al.* [BCRH15] combined PCA, Gauss map clustering, and agglomerative cluster analysis for edge detection. Some studies have incorporated statistical theories, such as normal distribution [WFDE13] and Poisson distribution [ZGW*16], into traditional geometric features for feature detection. Ahmed *et al.* [ATC*18] used an adaptive density-based threshold to differentiate edge points and then proposed a novel corner detector that combines curvature vectors and their geometrical statistics to detect corner points.

Several studies on feature extraction are also falling into the second category. Daniels *et al.* [DIOHS08] used RMLS to fit multiple surfaces to the neighbourhoods of the potential feature points and projected each point to its nearest intersection between the surfaces. Biosca *et al.* [BL08] proposed a clustering-based approach to capture feature lines on point clouds by planar segmentation. Hackel *et al.* [HWS16] proposed a two-stage discriminative learning-based method to extract the contours, i.e. lines along which the surface orientation sharply changes. Lee *et al.* [LB16] proposed a method that reconstructs feature curves based on the generation of developable surfaces.

Some researchers have also tried to detect features from preprocessed point clouds rather than on coordinate information directly. These techniques [GY19, SF04, HPW05] can capture fine features for more information than that provided by original models. However, most of these methods have yet to compute geometric invariant information, furthermore, both mesh reconstruction and voxelization induce large computational costs and may lead to the loss of some sharp features.

Deep-learning-based methods. With the rapid development of machine and deep learning, many researchers have also applied neural networks to detect features for point clouds. In early machine-learning-based methods, the input point clouds are transformed into regular voxel grids for CNN to process [QSN*16, DQN17], entailing high computational cost, and are generally limited by its resolution. Qi *et al.* [QSKG17] introduced PointNet to learn features for object classification and segmentation. Subsequently, they proposed an enhanced network with a hierarchical feature learning technique, referred to as PointNet++ [QYSG17]. The features extracted by PointNet and PointNet++ are relatively global features and unable to recognize fine features. Inspired by [QSKG17, QYSG17], Jiang *et al.* [JTYX18] proposed EC-Net, which has been designed to detect edges in point clouds and particularly adapted for sharp features; however, for tiny structures, the network is unable to obtain clear edges in point clouds. The EC-Net was improved by Loizou *et al.* [LAK20] to enhance its sensitivity to the boundaries between parts of a model, but the effect for sparse or tiny features is not satisfactory. To detect feature points, Wang *et al.* [WXX*20] designed PIE-Net, which treats point cloud curve inference as curve proposal followed by selection, which is sensitive to non-uniform sampling point clouds. Hu *et al.* [HZB*20] proposed JSENet for both semantic segmentation and semantic edge detection. Mostly recently, Himeur *et al.* [HLP*21] designed a network by proposing a new parameterization adding to each point a set of differential information on its surrounding shape reconstructed at different scales; and Bazazian *et al.* proposed [BP21] EDCNet, which uses both edge labels and segmentation labels as the Ground Truth.

α -shape. The α -shape was originally proposed in Edelsbrunner and Mücke [EM94]. In the 2D domain, α -shape has been used for several applications, such as shape reconstruction [MAM10] and boundary extraction [DSGC19]. However, in the 3D domain, it was mostly extended for reconstructing 3D surfaces [XH03]. Teichmann *et al.* [TC98] proposed modifying the original α value by point density estimation and manual intervention reduction. Similar to α -shape, Attali [Att98] proposed r-regular shape, which also makes use of balls to reconstruct shapes. In summary, several researchers have applied α -shape technique on surface reconstruction by triangulation point clouds or generating Voronoi graphs, resulting complex algorithms and an increase in computational cost.

LBP. Many filters may be used to complete the detection of feature lines in images. These filters include the LBP [OPH96, Pie10], a simple yet efficient texture operator, for example, Ahonen *et al.* [AHP04] used LBP for face recognition. LBPs, which do not directly handle 3D models, have been incorporated into point clouds in other studies by voxelizing point clouds or projecting point clouds from different views [ZWZ*18, MTB19, MTBDC20].

3. Method

This section presents the feature extraction method consisting of two main stages. The first stage improves the α -shape technique by generating two spheres with an adaptively computed radius as balls rolling on both the inner and outer surfaces of point clouds. Consequently, the potential feature points can be detected and the topologies of the neighbourhoods of points are created. The second stage generates SGLBPs for the potential feature points, which en-

code the topologies of the neighbourhoods. SGLBPs are ultimately analysed by defining six patterns and then non-feature points are iteratively removed from the potential feature points. The remaining points are identified as feature points. The overall procedure of our approach is illustrated in Figure 2.

3.1. Potential feature extraction and the topology construction

Two balls roll on the inner and outer surfaces of the point clouds but cannot reach the feature regions when roll on surfaces. Moreover, whether a point in the feature regions is definitely a feature point is mainly determined by their neighbouring information. Therefore, with these two observations considered, the α -shape is well suitable for both feature detection and neighbourhood description.

The α -shape is previously used to reconstruct the surface of point clouds in a large number of studies [EM94, TC98, XH03]; however, the main disadvantage of the α -shape is the difficulty of processing both whole shapes and their details. That is, the output of the α -shape is markedly different with respect to α values. With a small α , the reconstructed surfaces are not smooth and lead to additional fracture surfaces. By contrast, a large α value may lose more shape details on point clouds. For example, Figure 3 presents the results of α -shape with $\alpha = 0.10$ and 0.11 , respectively. Thus, in the current study, the α -shape is improved. Different from constructing the α -shape of the entire point cloud using a fixed α value, we compute different α values for different points, based on the local sampling densities of the point, which is proven effective and adaptive.

The improved α -shape is computed as follows. Given a point cloud $P = \{p_1, p_2, \dots, p_i, \dots, p_n\}$, for each point p_i , the improved α -shape uses Equation (1) to calculate α_i , where p_j is the k nearest neighbours that is denoted by Q_i . Then α_i is used as the radius of the ball, which is utilized to create the edges in G . Specifically, we randomly select two different points p_a and p_b in Q_i , then two spheres with radius α_i and passing points p_a, p_b and p_i , are generated. That is to say, points p_a, p_b , and p_i should be located on these two spheres, as shown in Figure 4. Given the two spheres, the edges of the points in Q_i can be computed. In particular, in the absence of other points $p_j \in Q_i$ located in the two spheres (Figure 4(a)), the points p_a, p_b and p_i are connected in G , denoted by orange lines in Figure 4(a). However, if at least one point $p_j \in Q_i$ is located in the two spheres, such as the green points in Figure 4(b), the points p_a, p_b and p_i are not considered connected. After any combination of p_a and p_b is checked using the proposed algorithm, the processing of point p_i is completed. Then another point p_{i+1} in P is chosen to repeat this process until all points in P are processed. Figure 3 also presents the results of the α -shape with $k = 5$ (the proposed method). It can be seen that the improved version of the α -shape is more adaptive to different models and can capture more details, compared with the traditional α -shape generated using a fixed α value.

$$\alpha_i = \frac{1}{k} \sum_{j=1}^k \|p_i - p_j\|_2 \quad (1)$$

Ideally, the points that the rolling balls cannot reach and are not involved in G are feature points. However, the computation of an appropriate radius of the ball that can accurately detect the feature points is non-trivial since the point cloud is usually non-uniformly

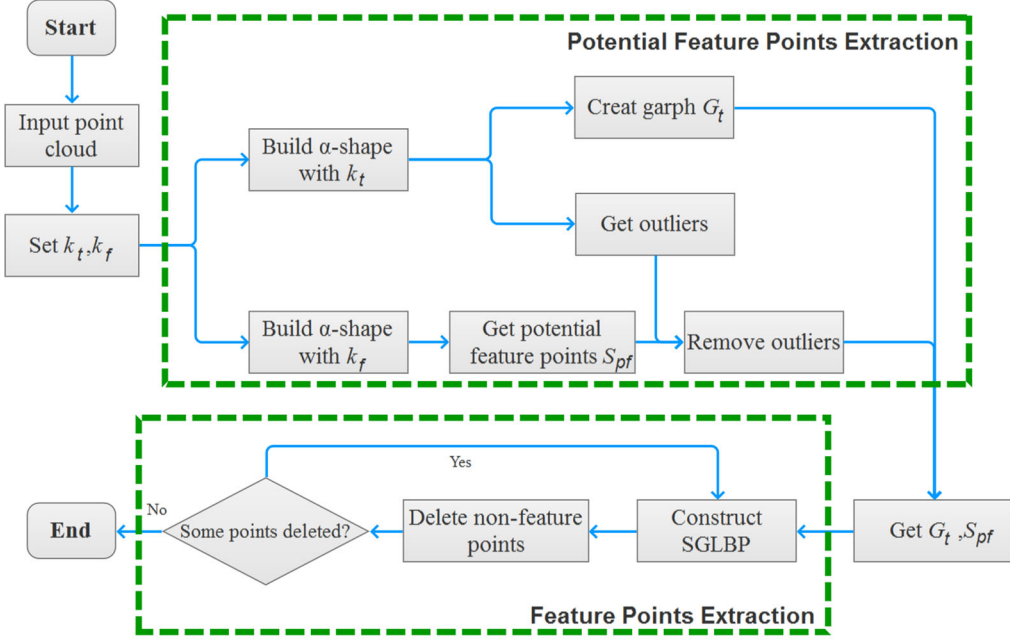


Figure 2: The flowchart of the proposed method.

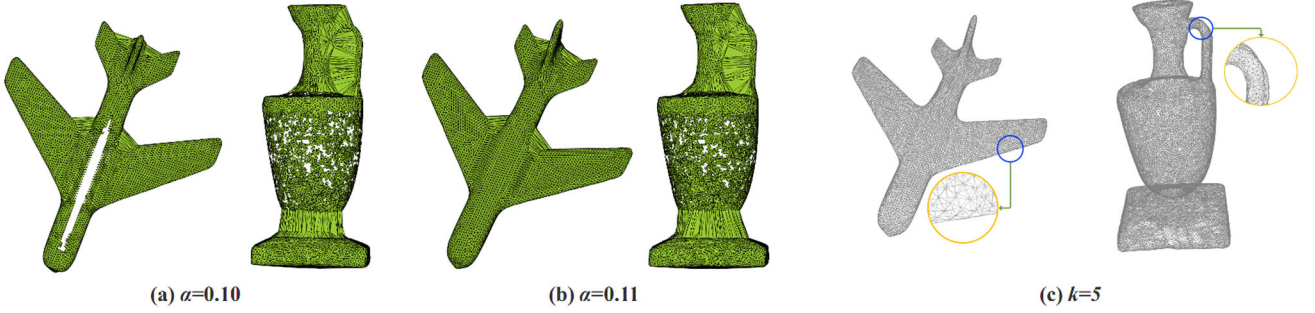


Figure 3: The illustration of the traditional α -shape and the improved α -shape. (a) Results of traditional α -shape with $\alpha = 0.10$; (b) results of traditional α -shape with $\alpha = 0.11$; (c) results of the improved α -shape with $k = 5$.

sampled and contains both sharp and smooth features. For example, Figure 5 shows how two rolling balls detect different potential feature points with different values of α_i (different values of k lead to different values of α_i , and different values of α_i lead to different G). With a small ball rolling on the surface in Figure 5(a), no feature point is captured, as shown in Figure 5(b), but the local topology of the neighbourhood is complete. An increase in k leads to an increase in α_i , but one feature point is detected and another is missing, as shown in Figure 5(c). When k continues to increase, both two feature points are detected. However, one non-feature point is also erroneously captured, and the topology of the neighbourhood is incomplete, as shown in Figure 5(d).

Therefore, we generate two α -shapes with different values of k —one α -shape G_t generated with a small k for representing the topology of the neighbourhood around a point like Figure 5(b) shows, and another α -shape G_f generated with a larger k is used to detect po-

tential feature points as many as possible, as shown in Figure 5(d). Thus the next step of the proposed method can effectively separate the real feature points by performing a further analysis on the α -shape G_t of the potential feature points that are obtained by G_f .

To generate the topology of the neighbourhood around the point, we construct graph G_t , which has as many connected nodes as possible, by setting k with a small value denoted by k_t . In case of a particularly large radius α_i , given that the outliers are far away from the main shape of point clouds, we redefine Equation (1) with a constraint as Equation (2):

$$\alpha_i = \begin{cases} 0 & \text{if } \eta \times \text{rate} < r \\ r & \text{if } \eta \times \text{rate} \geq r \end{cases} \quad \text{s.t. } r = \frac{1}{k_t} \sum_{j=1}^{k_t} \|p_i - p_j\|_2 \quad (2)$$

where η is the minimum radius generated by the algorithm in the previous iterations; we set $\eta = r$ in the initial stage, and $\text{rate} = 5$

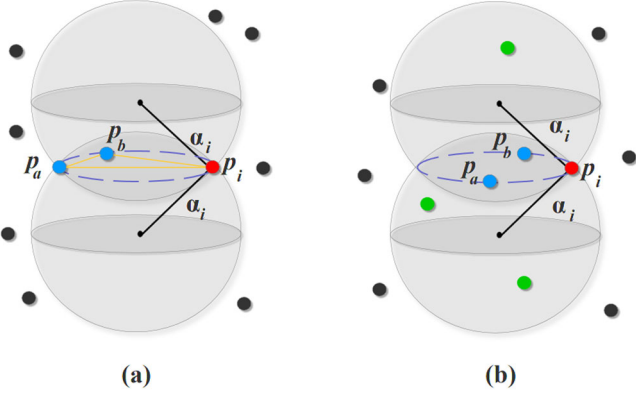


Figure 4: The illustration of the computation of edges in G . (a) Three points considered connected; (b) three points considered unconnected.

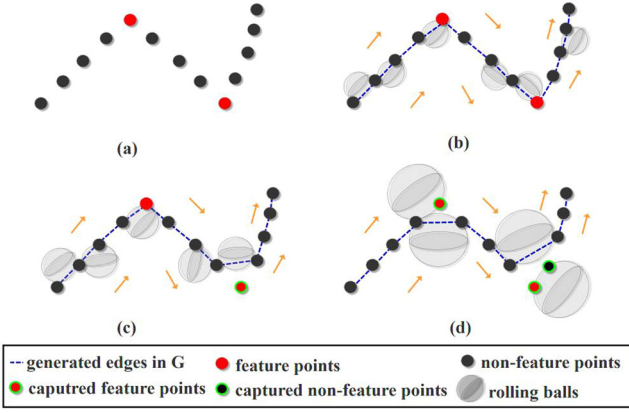


Figure 5: Two rolling balls detect different potential feature points with different values of α_i : (a) the given point cloud; (b) the results of small balls; (c) the results of medium balls; (d) the results of large balls.

in all the experiments except for the experiments on the highly non-uniformly sampled models (as shown in Figure 13) where $rate = 20$. This constraint leads to α_i set to 0 at outliers, which means those points will not be connected in G_i ; thus the outliers can be removed and will not be processed by subsequent steps.

G_f is generated by setting a larger k denoted by k_f , allowing more potential feature points to be captured in case the real feature points are missing. The points that the rolling balls cannot reach and are not involved in G_f are marked as potential feature points. Notably, the real feature points, noises, and some non-features are all considered as potential feature points at this stage, which comprise S_{pf} . The aforementioned process is summarized in Algorithm 1, and the captured potential feature points on a plane model and a vase model are shown in Figure 6(a).

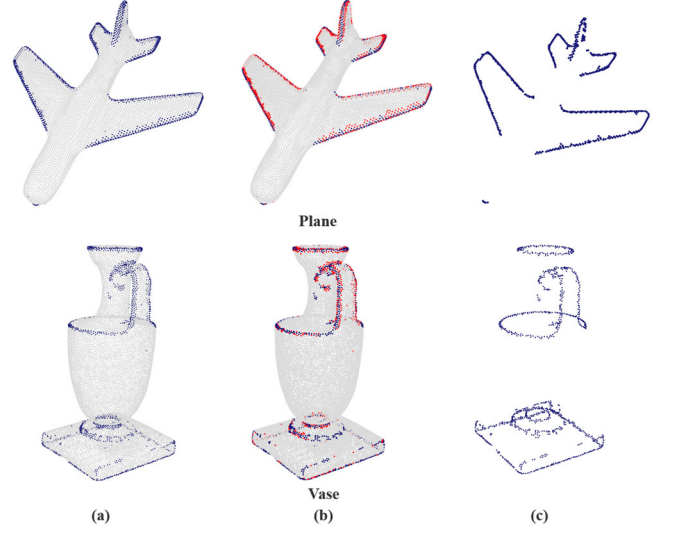


Figure 6: The results of potential feature points and feature points. (a) All captured potential feature points are marked in blue; (b) non-feature points are marked in red; (c) the final feature points captured by the proposed algorithm.

Algorithm 1. Construct an α -shape

Input: k : scale of neighborhood; $rate$: a threshold; P : point cloud;
Output: α -shape; G ; S ;

```

1: initial  $\eta$ , graph  $G$  and point set  $S$ ;
2: while exist an unvisited point in  $P$  do
3:   get an unvisited point  $p_i$ 
4:   set  $p_i$  visited
5:   calculate  $\alpha_i$  with Eq.2
6:   get  $k$  nearest neighbors  $Q_i$ 
7:   if  $\alpha_i == 0$  then
8:     continue
9:   end if
10:  if  $\eta > \alpha_i$  then
11:     $\eta = \alpha_i$ 
12:  end if
13:  for each point pair  $\langle p_a, p_b \rangle$  in  $Q_i$  do
14:    construct two spheres  $C$ 
15:    if no points of  $Q_i$  exist in  $C$  then
16:      connect  $p_a, p_b$  and  $p_i$  in graph  $G$ 
17:    end if
18:  end for
19: end while
20:  $S = P - getNodes(G)$ 

```

3.2. SGLBPs for describing the topology of neighbourhood

By performing the aforementioned process, the potential feature points are captured via an α -shape G_f , and the topologies of the neighbourhoods of potential feature points are constructed via an α -shape G_i . The topology of the neighbourhood of the potential

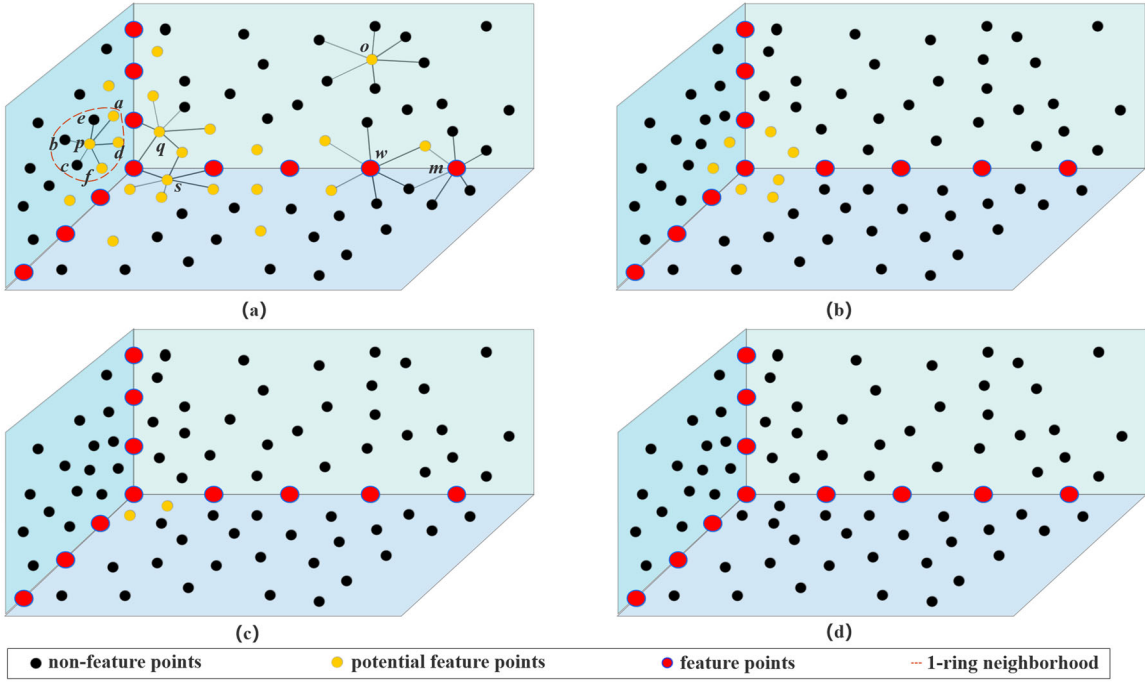


Figure 7: The illustration of the non-feature points removing process. (a) Both the yellow and red points are the potential feature points captured by the first stage, however, yellow points are non-feature points and only red points are the real feature points; (b) the result of the first iteration, which removes some non-feature points (the yellow points); (c) the result of the second iteration, which also removes some non-feature points (the yellow points); (d) the result of the third iteration, which removes all the non-feature points.

feature point p is represented by a subgraph G_i^p of G_i , which contains the 1-ring neighbouring points of p in G_i . Given that the neighbouring points of the potential feature points are either potential feature points or non-potential feature points, we adapt LBPs to encode the subgraph G_i^p , thus forming an SGLBP code for encoding the topology of the neighbourhood of p .

The generation of SGLBP contains two steps. First, given G_i^p , the 1-ring neighbouring points should be encoded in a linear sequence in the SGLBP code. To ensure the sequence of the neighbouring points, we encode the neighbouring points as follows. For example, the point p in Figure 7(a), its 1-ring neighbouring points are $\{a, b, c, d, e, f\}$. We start by randomly selecting one neighbouring point as a seed point, such as point a , and push it into an empty queue Q_c ; thus, Q_c becomes $\{a\}$. Then we select the point n such that $\angle apn$ is minimal, and push it into Q_c . In this iteration, the angle is $\angle ape$, and Q_c becomes $\{a, e\}$. Point e is then used as a new seed point, and the aforementioned point selection process is repeated. Q_c then becomes $\{a, e, b\}$ after the second iteration. The point selection process is terminated when all neighbouring points are pushed into Q_c . Finally, Q_c becomes $\{a, e, b, c, f, d\}$, as shown in Figure 7(a).

Then, the second step converts the Q_c into binary codes. Specifically, we let 1 denotes the potential feature points and 0 denotes the non-potential feature points, thereby forming the SGLBP code. For instance, the SGLBP of p in Figure 7(a) is $\{1, 0, 0, 0, 1, 1\}$. Then the sequence of SGLBP is again rearranged to ensure that the binary value of the code is the maximum permutation, and the code

becomes $\{1, 1, 1, 0, 0, 0\}$. The goal of this operation is to simplify the computation of N_{jump} , which counts the switch number from 0 to 1 or 1 to 0 in SGLBPs. Notably, the switch between the first and last element is not counted in N_{jump} . To illustrate, the switch number of $\{1, 1, 1, 0, 0, 0\}$ and $\{1, 1, 1, 0, 1, 0\}$ is 1 and 3, respectively.

3.3. Feature extraction based on SGLBPs

The potential feature points detected in the first stage contain feature points, non-feature points that are located near the feature points and some noisy points. Given the SGLBPs of potential feature points generated in the previous step, we present an iterative process to delete the non-feature points and noises from the potential feature points, retaining only the feature points.

We first define six patterns, denoted by PT_{0-0} , PT_{0-1} , PT_{1-A} , PT_{1-B} , PT_{1-C} and PT_2 respectively, to analyse SGLBPs on the basis of the value of N_{jump} , as illustrated in Figure 8. N_{jump} counts the switch number from 0 to 1 or 1 to 0 in SGLBPs.

Definition 1. Pattern PT_{0-0} indicates that N_{jump} of SGLBP is 0 and the SGLBP code of the pattern PT_{0-0} only contains 0. For instance, point o in Figure 7(a).

Definition 2. Pattern PT_{0-1} indicates that N_{jump} of SGLBP is 0 and the SGLBP code of the pattern PT_{0-1} only contains 1. For instance, point s in Figure 7(a).

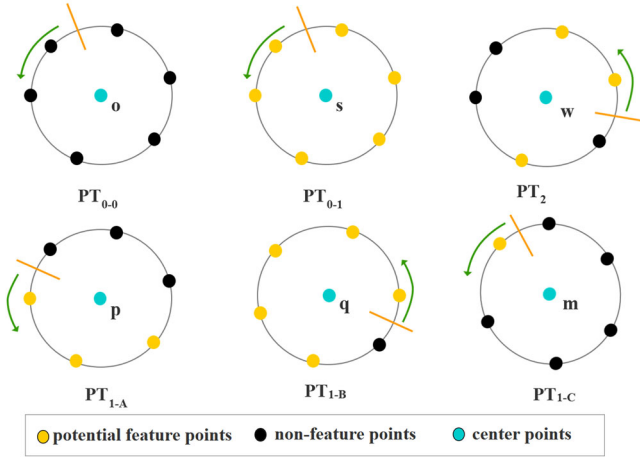


Figure 8: The illustration of the six different patterns of subgraph-based local binary patterns (SGLBPs).

Definition 3. Pattern PT_2 indicates that N_{jump} of SGLBP is greater than 1. For instance, point w in Figure 7(a).

Definition 4. Pattern PT_{1-A} indicates that N_{jump} is 1 and the SGLBP code contains similar numbers 0 and 1; furthermore, the difference between the number of 0 and 1 is only 1. For instance, point p in Figure 7(a).

Definition 5. Pattern PT_{1-B} indicates that N_{jump} is 1 and the SGLBP code contains more 1 than 0. And the difference between the number of 1 and 0 is larger than 1. For instance, point q in Figure 7(a).

Definition 6. Pattern PT_{1-C} indicates that N_{jump} is 1 and the SGLBP code contains considerably more 0 than 1. And the difference between the number of 0 and 1 is larger than 1. For instance, point m in Figure 7(a).

The proposed algorithm regards point A as a noisy point if its SGLBP code is classified into the pattern PT_{0-0} because no potential feature points exist around A, for example, point o in Figure 7(a). Correspondingly, point A is marked as a potential feature point if its SGLBP is classified into the pattern PT_{0-1} because it is surrounded by potential feature points, such as point s in Figure 7(a). Furthermore, points with pattern PT_2 , pattern PT_{1-B} and pattern PT_{1-C} types of SGLBP are also regarded as potential feature points, such as points w , q and m in Figure 7(a). However, point A with the PT_{1-A} -type of SGLBP can be a feature point or a non-feature point, such as point p in Figure 7(a). Thus, the SGLBPs of its 1-ring neighbouring points require further analysis. Supposing that the number of the 1-ring neighbours of point p is n_{1r} . If there are no more than $\omega \times n_{1r}$ points with the PT_{0-1} -type of SGLBP and the PT_{1-B} -type of SGLBP, point p is considered a non-feature point; and if there are more than $\omega \times n_{1r}$ points whose SGLBP codes are classified into PT_{0-1} and PT_{1-B} , the point p is still considered a potential feature point, and we empirically set $\omega = 0.2$.

Generally, by classifying the SGLBPs into the aforementioned six patterns, several potential feature points in S_{pf} are marked as non-feature points or noisy points and then removed from S_{pf} , as shown in Figure 7(b). However, some non-feature points in S_{pf} still exist as they are regarded as potential feature points in the first iteration, and the SGLBPs of the remaining points may change as several potential feature points are marked as non-feature points by the first iteration. Therefore, the aforementioned process should be repeated until no points are removed from S_{pf} , indicating that all non-feature points and noisy points are removed. The results of the 2nd and 3rd iterations are illustrated in Figures 7(c) and (d), respectively. Consequently, the remaining potential feature points in S_{pf} are identified as feature points extracted by the proposed method, as shown in Figure 7(d). Figure 6(b) shows those removed potential feature points and Figure 6(c) shows the final outputs. The iterative process of removing non-feature points is summarized in Algorithm 2.

Algorithm 2. Feature extraction

Input: S_{pf} : potential feature points;
Output: Feature points;

```

1: initial  $i = -1$ 
2: while  $i \neq 0$  do
3:    $i = 0$  //counting removed points for each loop
4:   for each  $p$  in  $S_{pf}$  do
5:     calculate SGLBP code and  $N_{jump}$  of  $p$ 
6:     classify  $p$  into the six patterns
7:     if  $p$  belongs to  $PT_{0-0}$  then
8:       remove  $p$  from  $S_{pf}$ 
9:        $i++$ 
10:    end if
11:    if  $p$  belongs to  $PT_{1-A}$  then
12:      get number  $n_{1r}$  of  $p$ 
13:       $num = \text{count}(PT_{0-1}) + \text{count}(PT_{1-B})$ 
14:      if  $num < \omega \times n_{1r}$  then
15:        remove  $p$  from  $S_{pf}$ 
16:         $i++$ 
17:      end if
18:    end if
19:  end for
20: end while

```

4. Experiment and Analysis

In this section, we first evaluate the performance of the proposed SGLBP by testing the sensitiveness to parameters and the robustness to varies levels of sampling densities and noises; then we compare our SGLBP with five existing methods, including traditional methods such as PBRG [ZGW*16], VCM [MOG11] and boundary estimation (BE) [RC11], and the deep-learning-based methods such as EC-Net [JTYX18] and PIE-Net [WXX*20].

For the PBRG, BE, EC-Net and PIE-Net, we use the default parameters suggested in the references. For VCM, the offset radius is set to 0.5 and the convolution radius is set to 0.25, as suggested in the reference; however, the parameter τ , which separates the edge points, is fine-tuned and in the Section 4.2.2, we listed the results obtained by setting $\tau = \{0.12, 0.17, 0.22\}$, respectively. For the

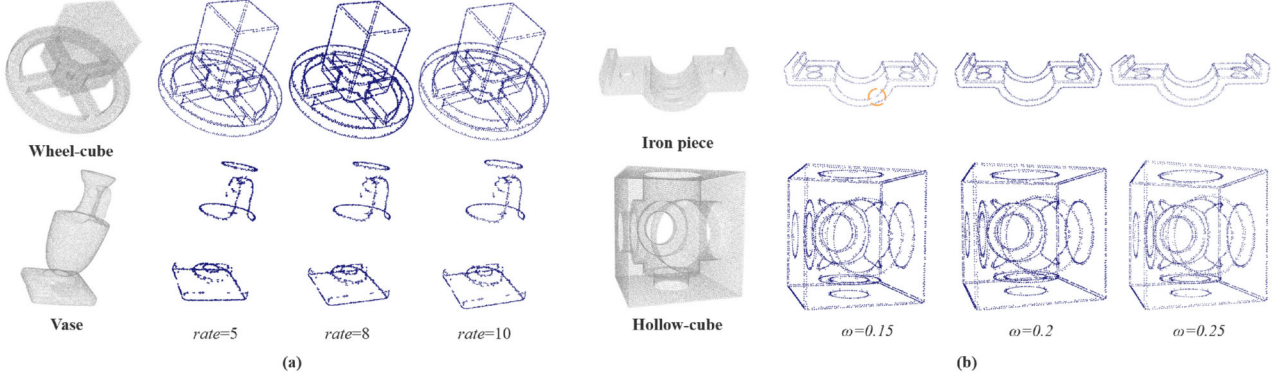


Figure 9: The results of feature extraction with different rate and ω . (a) The results of feature extraction with different rate; (b) the results of feature extraction with different ω .

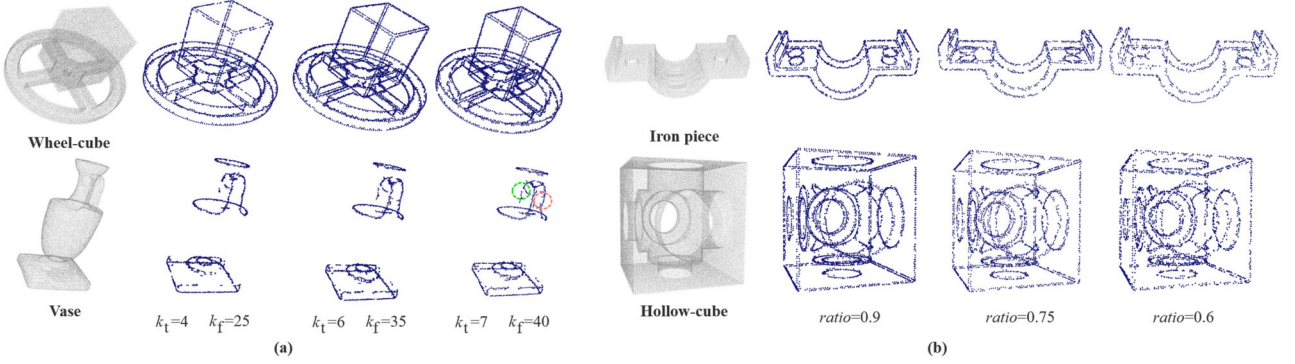


Figure 10: The results of feature extraction with different k and on models with different levels of sampling densities. (a) The results of feature extraction with different k_t and k_f ; (b) the results of feature extraction on models with different levels of sampling densities.

proposed SGLBP, we set $k_t = 5$, $k_f = 28$, $rate = 5$ and $\omega = 0.2$ as the default parameters and apply them on all models presented in the current study, except for the example in Figures 9 and Figure 10(a), which are used to demonstrate the influence of parameters. Furthermore, we set $rate = 20$ for the highly non-uniformly sampled models in Figure 13.

The codes of PBRG, EC-Net and PIENet that are provided by the authors, the BE implemented in PCL and the VCM implemented in CGAL are exploited to detect the features for testing. Furthermore, the data used in the experiments are from the Stanford 3D Scanning Repository, which contains real-scanned models, and the ABC data set, which contains both models and the corresponding Ground Truth [KMJ*19]. In our experiments, the point clouds are downsampled 8096 points by uniform sampling (implemented in open3D) on the original mesh shapes for PIENet, and the original point clouds without any sampling operation are utilized as the input for SGLBP, PBRG, BE, VCM and EC-Net.

4.1. Robustness

To test the robustness of the proposed method, we first set different $rate$, ω and k to evaluate the influence of these parameters on feature

extraction. Then the robustness to sampling density and Gaussian noises of the proposed method are evaluated.

4.1.1. Sensitivity to the parameters

In this section, we first use different values of $rate$, ω and k to extract features on different models and thereby evaluate the sensitivity of the proposed algorithm to the parameters. This part excludes scenarios in which the parameters are too small or too large.

Firstly, we evaluate the proposed method on Wheel-cube and Vase with different $rate$, and the value of $rate$ is increased from 5 to 10. As shown in Figure 9(a), the change of the $rate$ does not affect the results considerably. Furthermore, we set three different values of ω , i.e. varying from 0.15 to 0.25, to evaluate the robustness of SGLBP. It can be seen from Figure 9(b), when ω is set to 0.15, a little bit of features are missed (such as the part marked by the orange circle), but most of the features are captured successfully. With ω increased, the captured features become more complete.

As shown in Figure 10(a), for the Wheel-cube, the change in k_t and k_f does not considerably influence the feature points. That is, for sharp features, the feature points are the same despite the different

Table 1: The detailed information of downsampled models.

Model	N_{fn}	$ratio = 0.9$		$ratio = 0.75$		$ratio = 0.6$	
		N_{fd}	r_c	N_{fd}	r_c	N_{fd}	r_c
Iron piece	1531	1367	0.89	1236	0.81	1001	0.65
Hollow-cube	3667	3219	0.88	2864	0.78	2459	0.67

scales of the neighbourhoods. Furthermore, for the smooth features, when k_t and k_f increase, more details of the model are captured—for instance, the Vase gap becomes clearer when k_t is increased from 4 to 7 and k_f is increased from 25 to 40 (such as the part marked by the green circle in Figure 10(a)); however, some non-feature points are erroneously recognized as feature points, such as the points around the handle of the Vase (marked by the orange circle in Figure 10(a)), but the results are still satisfactory.

4.1.2. Robustness to the sampling density

In this part, we use different point clouds with different sampling densities to verify the robustness to the sampling density of the proposed method. The tested models are downsampled with different *ratio*, where *ratio* is the rate of change in the number of points between the original model and the downsampled model. Equation (3) shows the formula to calculate *ratio*:

$$ratio = \frac{N_s}{N_o} \quad (3)$$

where N_s is the number of points in the downsampled model, and N_o represents the number of points in the original model.

The change in feature points at different down-sampling levels with different *ratio* is shown in Figure 10(b). With different values of *ratio*, the outcome of the proposed method only slightly changes. For quantitative analysis, we also define a new metric r_c as follows:

$$r_c = \frac{N_{fd}}{N_{fn}} \quad (4)$$

where N_{fd} represents the number of feature points in different down-sampled models, and N_{fn} represents the number of feature points

in the original models. We thus count the number of points in the downsampled models, and the proportions of feature points between different models are calculated. The data are listed in Table 1. The values of r_c are basically equal to the corresponding *ratio*, confirming that the proposed method is robust to diverse levels of the sampling densities.

4.1.3. Robustness to noises

To evaluate the effect of noises on the results of feature point extraction, we define a disturbance radius to generate different scales of Gaussian noises for different models, such as $0.01d_B$, $0.02d_B$ and $0.03d_B$ (where d_B is the length of the model bounding box diagonal). The larger the disturbance of the radius, the more disorderly the model. The visible results are presented in Figure 11, which reveals that under different levels of Gaussian noises, the proposed method still can detect most feature points.

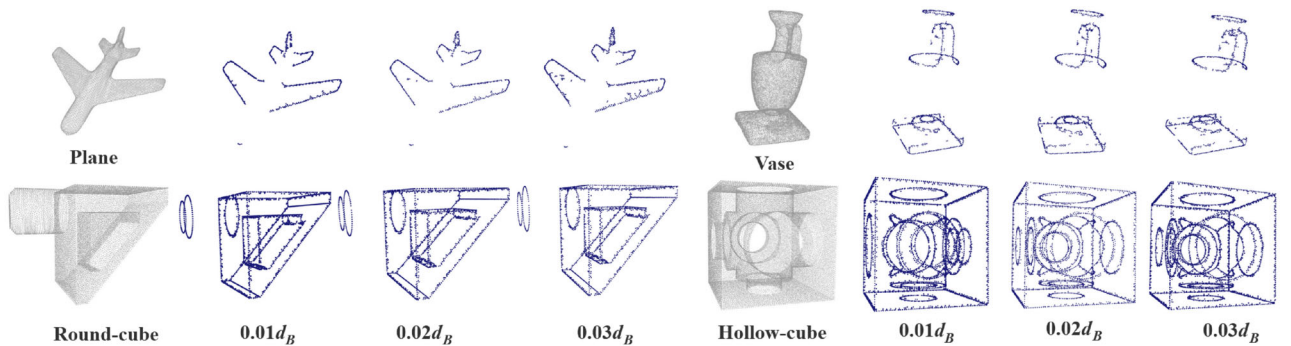
4.2. Results and comparisons

In this section, we compare the feature extraction effectiveness of the proposed method with that of the five existing methods from both qualitative and quantitative perspectives.

4.2.1. The qualitative evaluation

In this subsection, we first test those methods on four typical models, including a Plane model with more smooth features, a Wheel-cube model with more sharp features, a Vase model with both smooth and sharp features and an Iron piece model with more sharp features.

As shown in Figure 12(a), most features of the Plane are recognized using the proposed method, indicating the satisfactory performance on smooth features. It also can be seen from Figure 12(a) that PIE-Net can capture some of the edge points from the Plane, but most feature points are missed; furthermore, although EC-Net can deal with the smooth features, many features around the corner are missed. PBRG and BE produce similar results since they detect both feature points and some non-feature points that closely near the features. However, VCM fails to deal with the smooth features. Figure 12(b) shows that VCM can successfully capture the edges on the Wheelcube model, but numerous non-feature points

**Figure 11:** The results of feature extraction on models disturbed by different levels of Gaussian noises.

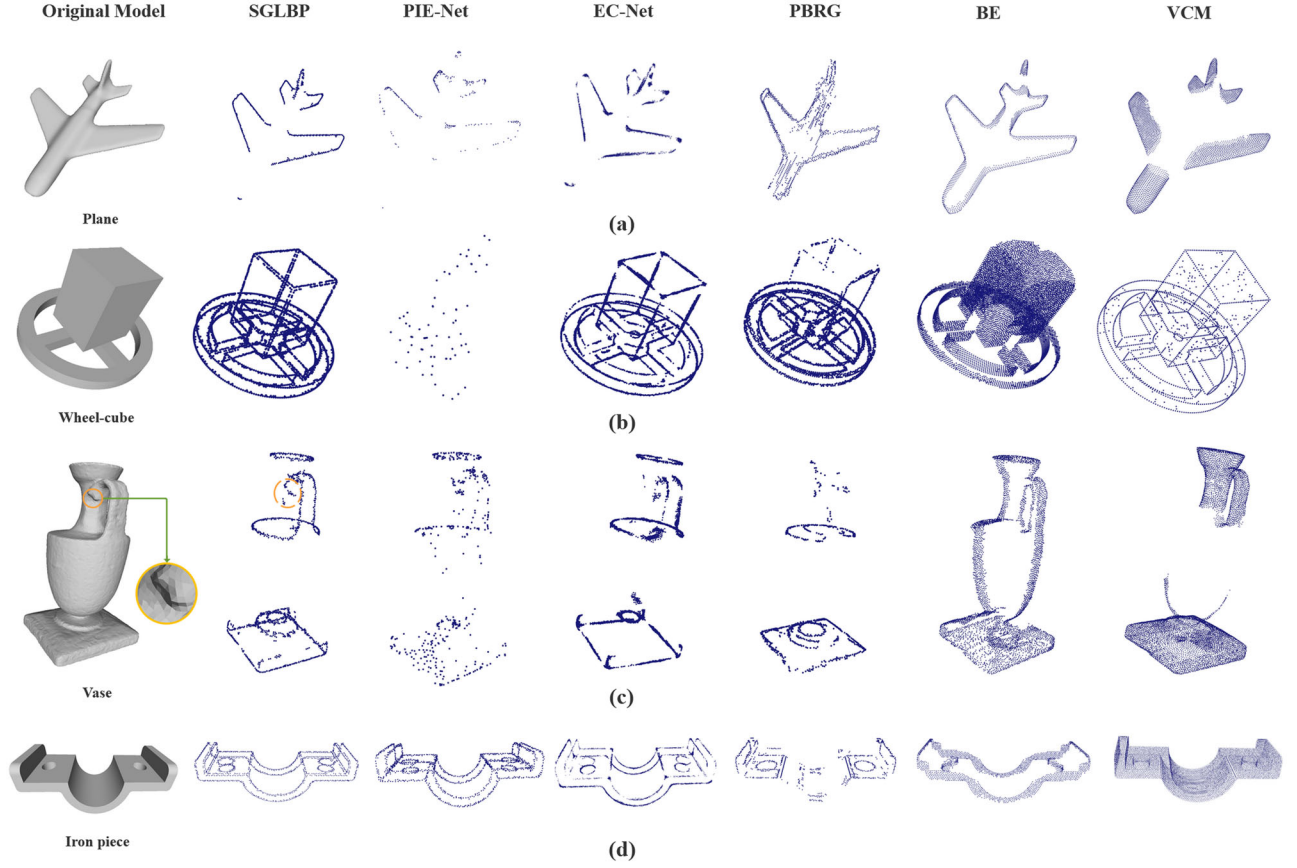


Figure 12: The results of feature extraction on four models with different features. The results of VCM are the best results when τ varies from 0.1 to 0.3.

are also erroneously captured; furthermore, both PIE-Net and BE cannot handle this model. Although SGLBP, EC-Net and PBRG all perform well on sharp features (such as the base part), however, SGLBP still achieves better results. As shown in Figure 12(c), the Vase model contains both smooth and sharp features, and it also contains some tiny features. It can be seen that the proposed SGLBP significantly outperforms the other methods since it is sensitive to the tiny features and produces a result with more details (such as the part marked in the orange circle). For the Iron piece model, as shown in Figure 12(d), SGLBP, PIE-Net and EC-Net all successfully capture the feature points whereas the other methods cannot handle this model, however, compared to EC-Net, SGLBP and PIE-Net, our SGLBP produces clearer edges.

Although all models used in this study exhibit varying degrees of non-uniform sampling, these adverse conditions are still not sufficiently strong to influence the outcome of feature point extraction. Thus, to further test the robustness to non-uniform sampling, we also use two other models, which exhibit highly non-uniform sampling densities (whose main bodies contain only a few points and whose heads have more points). The results in Figure 13 reveal that compared to other methods, the proposed algorithm exerts a significantly better effect, demonstrating the effectiveness and robustness of the proposed SGLBP.

Furthermore, we also test those methods on two complex and real-scanned models from the Stanford 3D Scanning Repository, i.e. the Dragon and Lucy models. Moreover, both the Dragon and Lucy models contain many tiny features. PIE-Net, PBRG and VCM fail to produce an effective output and Figure 14 shows the results of SGLBP, EC-Net and BE. As shown in Figure 14, compared to EC-Net and BE, the proposed SGLBP can capture more tiny features, such as the head part of the dragon model, and the face part, wings part and the feet part of the Lucy Model.

4.2.2. The quantitative evaluation

In this part, we compare the outputs of the six methods on 6000 point clouds in ABC data set, which all provide the Ground Truth. In order to quantitatively evaluate the effectiveness, we use seven metrics to quantify the performance of these algorithms. Note that the features extracted by EC-Net are fitted points instead of the original feature points on point clouds, so we first normalized the outputs of these methods and Ground Truth. Furthermore, the corresponding point pairs are calculated using ICP instead of Euclidean distance in Moscoso Thompson *et al.* [MTAM*19], which obtains higher accuracy.

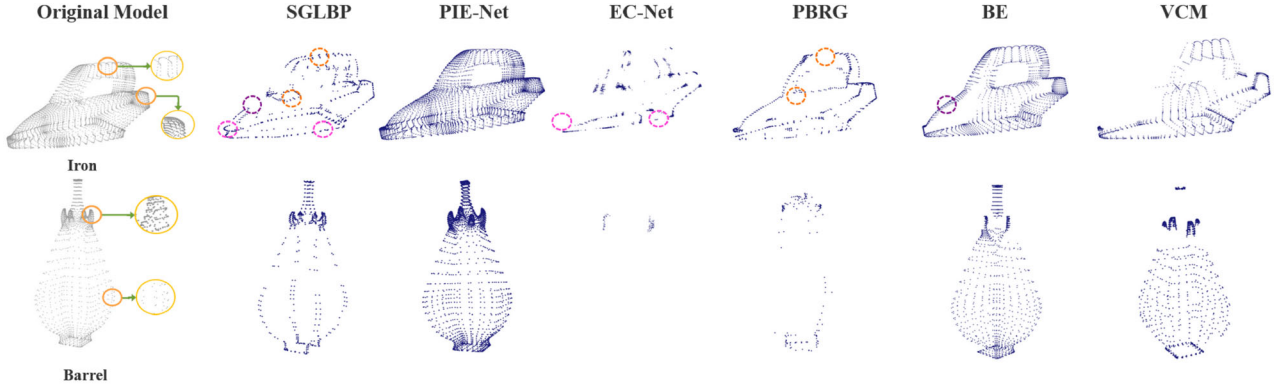


Figure 13: Results of feature extraction on highly non-uniform sampling models. The results of VCM are the best results when τ varies from 0.1 to 0.3.

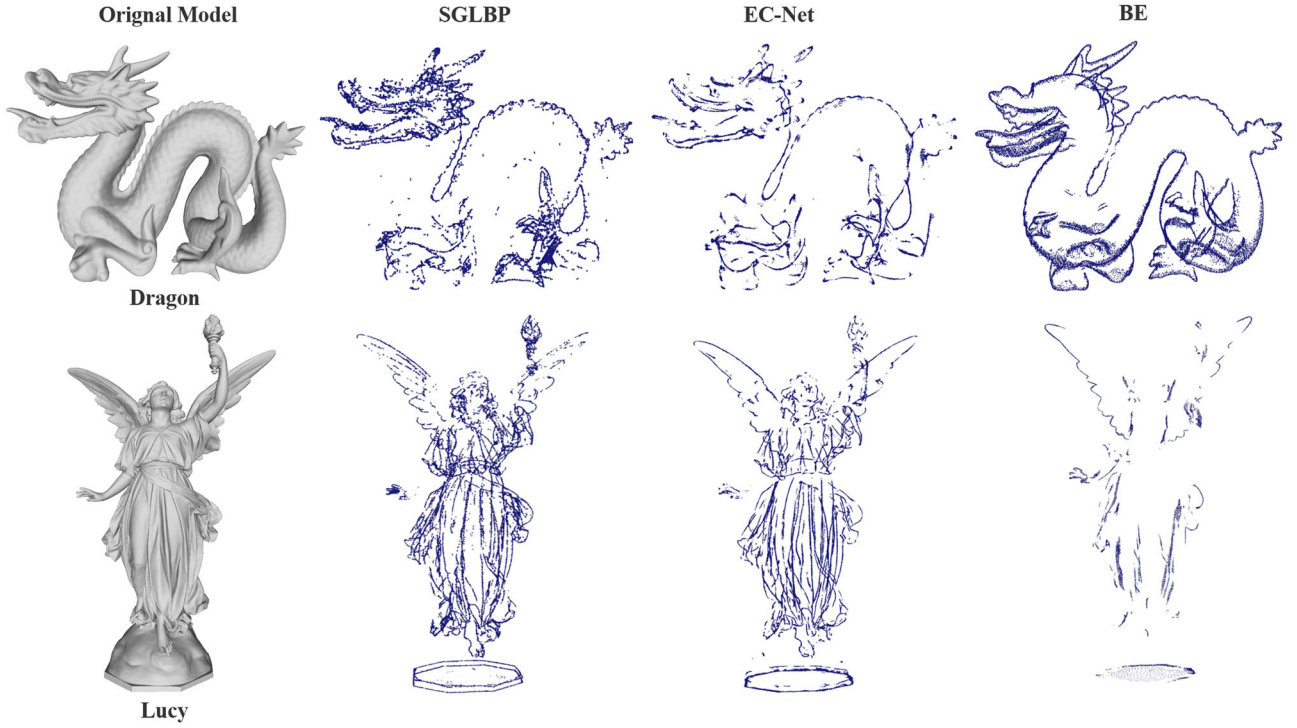


Figure 14: Results of feature extraction on Dragon model and Lucy model.

The first metric is *Jaccard* as defined in Equation (6) [MTAM*19], which is utilized to evaluate how well the features extracted by the participants fit the Ground Truth and *vice versa*:

$$dice(P, GT) = \frac{2|P \cap GT|}{|P| + |GT|} \quad (5)$$

$$Jaccard(P, GT) = \frac{dice(P, GT)}{2 - dice(P, GT)} \quad (6)$$

P is the output of each method while $|P|$ represents the points' number in P . GT is the Ground Truth from ABC data set and $|GT|$ repre-

sents the points' number in GT . $|P \cap GT|$ represents the number of the corresponding point pairs from P and GT , which are achieved by ICP. Furthermore, the precision and recall are calculated by Equations (7) and (8), respectively.

More precisely, we also utilize two other metrics presented in [HLP*21] to further evaluate and compare these methods, i.e. the matthews correlation coefficient (*MCC*) (Equation (9)) and the Intersection over union score (*IoU*) (Equation (10)), which are calculated based on the values of true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN). We regard the corresponding points achieved by ICP as TP. Furthermore, inspired by Ao

Table 2: The Jaccard, Precision, Recall, MCC, IoU and $RP(\delta)$ scores of the six methods.

Method	Jaccard	Precision	Recall	MCC	IoU	$RP(0.3)$	$RP(0.25)$
SGLBP	0.2574	0.3497	0.5428	0.275	0.2545	0.5815	0.6464
PIE-Net	0.0703	0.2977	0.1067	0.1605	0.0703	0.0008	0.0037
EC-Net	0.0688	0.0813	0.421	-0.0844	0.0663	0	0.0002
PBRG	0.2079	0.3491	0.4076	0.2608	0.2062	0.4104	0.557
BE	0.1636	0.2442	0.4663	0.1822	0.165	0.2039	0.2715
VCM ($\tau = 0.12$)	0.1172	0.1879	*0.9438	0.1702	0.1784	0.1842	0.2309
VCM ($\tau = 0.17$)	0.2095	0.2393	*0.905	0.2054	0.2122	0.2385	0.2877
VCM ($\tau = 0.22$)	0.2336	0.2891	*0.8615	0.2562	0.239	0.2833	0.3282

The best scores are marked in bold, and * denotes the ineffective score.

et al. [AHY*21], $RP(\delta)$, which is calculated using Equation (11), is introduced.

$$Precision = \frac{TP}{|P|} \quad (7)$$

$$Recall = \frac{TP}{|GT|} \quad (8)$$

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (9)$$

$$IoU = \frac{TP}{TP + FP + FN} \quad (10)$$

$$RP(\delta) = \frac{\sum_{i=1}^N n_i}{N} \quad (11)$$

$$s.t. \begin{cases} n_i = 1 & \text{if } Precision_i > \delta \ \& \ Recall_i > \delta \\ n_i = 0 & \text{if } Precision_i \leq \delta \ \parallel \ Recall_i \leq \delta \end{cases}$$

where N is the number of the tested point clouds. The MCC is a correlation between the observed and predicted binary classification and ranges from $[-1, 1]$, which shows whether the prediction is good. The IoU specifies the overlap between the P and GT ; and the $RP(\delta)$ measures how many models in the tested dataset are successfully processed. And all the metrics are expected to 1.

The Jaccard, Precision, Recall, MCC, IoU, $RP(0.3)$ and $RP(0.25)$ scores produced by these methods are listed in Table 2. It can be seen that EC-Net produces the lowest Jaccard, Precision, MCC, IoU, $RP(0.3)$ and $RP(0.25)$ scores since the feature points extracted by EC-Net are fitted points instead of the original feature points on point clouds, however, the Recall of EC-Net is higher than that of PIE-Net and PBRG. PIE-Net also produces a relatively lower performance since the results are considerably affected by the down-sampling operation; moreover, another drawback of the PIE-Net is that the input point clouds must be downsampled same number of points. From Table 2, it also can be found that the proposed SGLBP outperforms the PBRG and BE for all the metrics, and the Jaccard, Precision, MCC, IoU, $RP(0.3)$ and $RP(0.25)$ of our SGLBP are also higher than that of the VCM. The Recalls of the VCM are significantly higher than that of the other methods, however, the recall scores of VCM are ineffective. This is because the computation of

the recall only considers the TP, and FP (erroneously extracted non-features) is not considered. Thus, the worst result that all points on point clouds are extracted as features can still lead to $recall = 1$. For example, the unsatisfactory results of VCM shown in Figure 12 can achieve higher recalls since all the features are captured, although more non-features are also erroneously extracted. In summary, the best performance all attribute to SGLBP, highlighting the high effectiveness, adaptiveness and accuracy of the proposed method. Figure 15 also presents some results of the proposed SGLBP on the ABC dataset.

4.3. Efficiency

In this subsection, we compare the efficiency of EC-Net, PBRG, BE, VCM and the proposed method. Since the input point clouds of PIE-Net are all downsampled 8096 points, the efficiency of PIE-Net is not tested in this subsection. Testing is consistently conducted using a computer with the following specifications: GPU, NVIDIA RTX 2080; CPU, AMD Ryzen 7 3700X and memory, 16G. For the characteristic of deep learning, we only record the testing time of EC-Net, and training time is not considered. For PBRG, BE and VCM, we record their total running time for different models.

We use six typical models to intuitively evaluate the efficiency of the methods, i.e. two models (such as Plane and Vase) are small and simple, two models (such as Hollow-cube and Dargon) are medium and the other two models (such as Lucy1 and Lucy2) are large. The detailed information is listed in Table 3, and ∞ means no available results are obtained. As shown in Table 3, the proposed method is markedly more efficient than PBRG, particularly for large models. PBRG requires hundreds of seconds, whereas the proposed method only uses dozens of seconds. The reason is that PBRG needs to use data based on differential geometry and some features are visited by multiple times in PBRG, which both involve a large amount of calculation. When dealing with models with less than 100,000 points, the SGLBP is still faster than BE and VCM; however, when the scales of the models increase, the proposed method occupies more time than BE and VCM. When compared with EC-Net on small models (such as Plane and Vase), the proposed method is slightly better than EC-Net. Although the EC-Net is more efficient than the proposed method on large models, the training stage does need more computational time, data and even hardware.

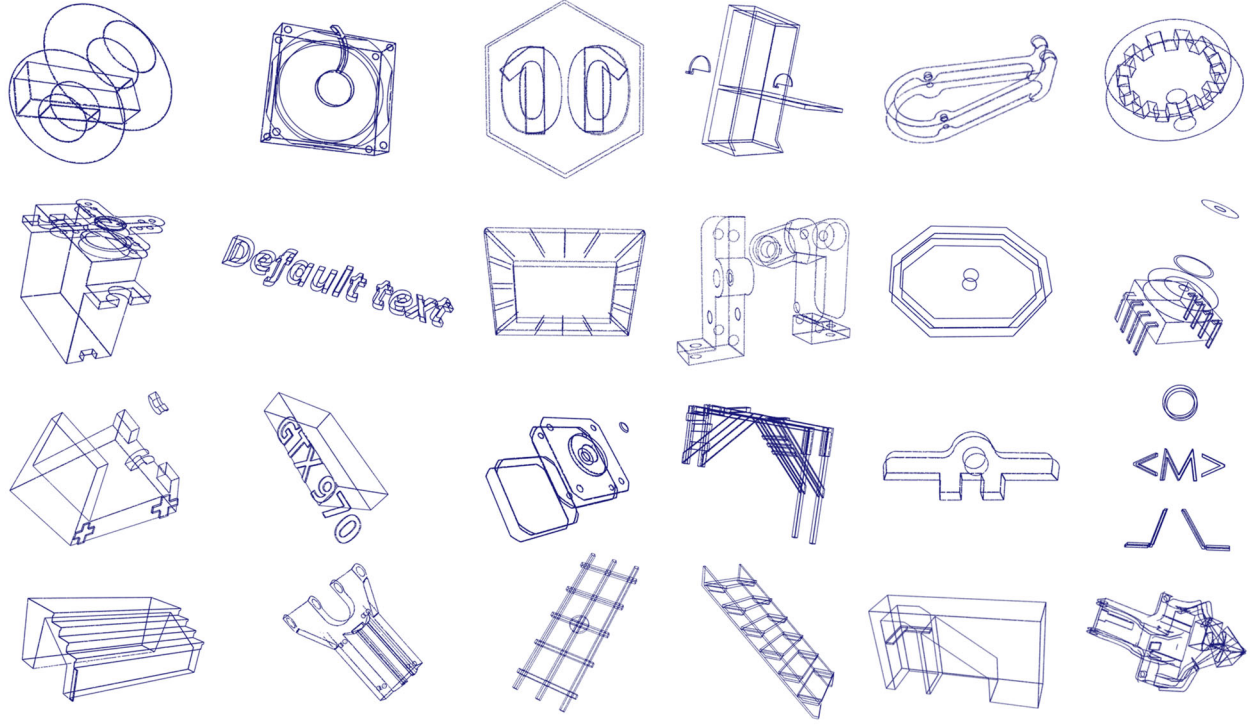


Figure 15: Results of feature extraction on ABC data set.

Table 3: The execution time of the methods on different scales of models.

Model	Point number	Runing times(s)				
		SGLBP	EC-Net	PBRG	BE	VCM
Plane	7651	1.211	<i>1.476</i>	13.048	1.81	9.109
Vase	14476	2.74	<i>2.933</i>	42.19	5.342	17.905
Hollow-cube	32169	<i>12.05</i>	6.493	868.396	23.732	31.687
Dragon	100207	<i>73.822</i>	24.076	∞	392.356	1093.688
Lucy1	328287	1371.45	71.197	∞	33.631	<i>1020.566</i>
Lucy2	1371187	31400.3	302.645	∞	634.267	4506.385

The best scores are marked in bold, and the second best scores are marked in *italic*.

4.4. Limitations

The proposed SGLBP may fail when handling the features that are located at only one surface, such as the boundary of a planar plane; moreover, the SGLBP may also fail on the severe anisotropic sampled models on which the topology reconstruction may fail. Furthermore, when dealing with large scale models (containing more than one million points), the efficiency of the proposed SGLBP significantly decreases.

5. Conclusion and Future Work

In this study, we propose a novel method for feature extraction from raw point clouds. First, two graphs, which can detect the potential feature points and describe the topology of the neighbourhoods, are

generated using an improved α -shape technique. The SGLBPs are then formed on the subgraphs generated in the first step, which encode the topologies of the neighbourhoods in binary vectors. Six useful patterns for analysing SGLBPs are ultimately presented, thus the non-feature points can be detected and removed. The potential feature points that remain are regarded as feature points extracted by the proposed method. As shown in Section 4, the proposed method achieves satisfactory performance on various models and exhibits robustness to Gaussian noises and sampling densities of varying degrees. Compared with other methods, the proposed technique exhibits universality because it can process non-uniform sampling models.

For the future research, we intend to allow the proposed algorithm to automatically choose different k values for different models, such as changing k , by calculating the local sampling density. The manner

by which suitable k nearest neighbours are to be selected to construct an α -shape is a point to consider. In addition, in order to promote the efficiency when dealing with large scale point clouds, performing graph convolutional networks on the α -shape graph to detect feature points is also an interesting and meaningful research point.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (NSFC) (61802311, 61772421, 61731015, 61802244, 61602380), Key R&D Program of Shaanxi Province (2019SF-272), Natural Science Basic Research Plan in Shaanxi Province of China (2019JQ-668) and the Fundamental Research Funds for the Central Universities (GK201903114).

References

- [AHP04] AHONEN T., HADID A., PIETIKÄINEN M.: Face recognition with local binary patterns. In *Computer Vision—ECCV 2004. Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg (2004), pp. 469–481. https://doi.org/10.1007/978-3-540-24670-1_36.
- [AHY*21] AO S., HU Q., YANG B., MARKHAM A., GUO Y.: Spinnet: Learning a general surface descriptor for 3D point cloud registration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (Virtual-only Conference, June 2021), pp. 11753–11762. <https://arxiv.org/abs/2011.12149>.
- [ATC*18] AHMED S. M., TAN Y. Z., CHEW C. M., MAMUN A. A., WONG F. S.: Edge and corner detection for unorganized 3d point clouds with application to robotic welding. In *Proceedings of the IEEE* (Oct. 2018), pp. 7350–7355. <https://doi.org/10.1109/IROS.2018.8593910>.
- [Att98] ATTALI D.: r -regular shape reconstruction from unorganized points. *Computational Geometry: Theory and Applications* 10, 4 (1998), 239–247. Special Issue on Applied Computational Geometry.
- [BCRH15] BAZAZIAN D., CASAS J. R., RUIZ-HIDALGO J.: Fast and robust edge extraction in unorganized point clouds. In *Proceedings of the 2015 International Conference on Digital Image Computing: Techniques and Applications (DICTA)* (Adelaide, SA, Australia, 2015), pp. 1–8. <https://doi.org/10.1109/DICTA.2015.7371262>.
- [BL08] BIOSCA J. M., LERMA J. L.: Unsupervised robust planar segmentation of terrestrial laser scanner point clouds based on fuzzy clustering methods. *ISPRS Journal of Photogrammetry and Remote Sensing* 63, 1 (2008), 84–98.
- [BP21] BAZAZIAN D., PARÉS M. E.: EDC-Net: Edge detection capsule network for 3D point clouds. *Applied Sciences* 11, 4 (Feb. 2021), 1833.
- [DHL*20] DENG Y., HAN S., LI J., RONG J., FAN W., SUN T.: The design of tourism product CAD three-dimensional modeling system using VR technology. *PloS One* 15, 12 (Dec. 2020), e0244205.
- [DIOHS08] DANIELS II J., OCHOTTA T., HA L. K., SILVA C. T.: Spline-based feature curves from point-sampled geometry. *The Visual Computer* 24, 6 (Apr. 2008), 449–462.
- [DM14] DIGNE J., MOREL J. M.: Numerical analysis of differential operators on raw point clouds. *Numerische Mathematik* 127, 2 (2014), 255–289.
- [DQN17] DAI A., QI C. R., NIESSNER M.: Shape completion using 3D-encoder-predictor CNNs and shape synthesis. In *Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2017), pp. 6545–6554. <https://doi.org/10.1109/CVPR.2017.693>.
- [DSGC19] DOS SANTOS R. C., GALO M., CARRILHO A. C.: Extraction of building roof boundaries from lidar data using an adaptive alpha-shape algorithm. *IEEE Geoscience and Remote Sensing Letters* 16, 8 (2019), 1289–1293.
- [DVC18] DIGNE J., VALETTE S., CHAINE R.: Sparse geometric representation through local shape probing. *IEEE Transactions on Visualization and Computer Graphics* 24, 7 (July 2018), 2238–2250.
- [DVVR07] DEMARSIN K., VANDERSTRAETEN D., VOLODINE T., ROOSE D.: Detection of closed sharp edges in point clouds using normal estimation and graph theory. *Computer-Aided Design* 39, 4 (2007), 276–283.
- [DWXW21] DAI J., WEI M., XIE Q., WANG J.: Aircraft seam feature extraction from 3D raw point cloud via hierarchical multi-structure fitting. *Computer-Aided Design* 130 (2021), 102945.
- [EM94] EDELSBRUNNER H., MÜCKE E. P.: Three-dimensional alpha shapes. *ACM Transactions on Graphics* 13, 1 (Jan. 1994), 43–72.
- [GB21] GÜNEN M. A., BEŞDOK E.: Comparison of point cloud filtering methods with data acquired by photogrammetric method and RGB-D sensors. *International Journal of Engineering and Geosciences* 6, 3 (2021), 125–135.
- [GWM01] GUMHOLD S., WANG X., MACLEOD R.: Feature extraction from point clouds. In *Proceedings of the 10th International Meshing Roundtable* (Newport Beach, CA, USA, 2001).
- [GY19] GAO Q., YAMAGUCHI Y.: Extraction of coherent and smooth feature lines from meshes with fine details. *Computers & Graphics* 82 (2019), 222–231.
- [GZWW19] GOJCIC Z., ZHOU C., WEGNER J. D., WIESER A.: The perfect match: 3D point cloud matching with smoothed densities. In *Proceedings of the 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (Long Beach, CA, 2019), pp. 5540–5549. <https://doi.org/10.1109/CVPR.2019.00569>.

- [GZZ21] GONG M., ZHANG Z., ZENG D.: A new simplification algorithm for scattered point clouds with feature preservation. *Symmetry (Basel)* 13, 3 (Feb. 2021). <https://doi.org/10.3390/sym13030399>.
- [HLP*21] HIMEUR C. E., LEJEMBLE T., PELLEGRINI T., PAULIN M., BARTHE L., MELLADO N.: PCEDNet: A lightweight neural network for fast and interactive edge detection in 3D point clouds. *ACM Transaction on Graphics* 41, 1 (2021), 1–21.
- [HPW05] HILDEBRANDT K., POLTHIER K., WARDETZKY M.: Smooth feature lines on surface meshes. In *SGP'05: Proceedings of the third Eurographics symposium on geometry processing* (Vienna, Austria, 2005), pp. 85–90. <https://doi.org/10.2312/SGP/SGP05/085-090>.
- [HWS16] HACKEL T., WEGNER J. D., SCHINDLER K.: Contour detection in unstructured 3D point clouds. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (Las Vegas, US, June 2016), pp. 1610–1618. <https://doi.org/10.1109/CVPR.2016.178>.
- [HZB*20] HU Z., ZHEN M., BAI X., FU H. & TAI C. European Conference on Computer Vision (2020), pp. 222–239. https://doi.org/10.1007/978-3-030-58565-5_14.
- [JTYX18] LEQUAN Y., XIANZHI L., CHI-WING F., DANIEL C. O. PHENG-ANN H. Ec-net: an edge-aware point set consolidation network. In *Proceedings of the European conference on computer vision (ECCV)* Munich, Germany, 2018), pp. 386–402.
- [JYH*13] JING C., YUAN C., HU S., YAN Y., LIANG X.: Feature lines extraction for remote rendering on point model. *Computing and Visualization in Science* 15, 3 (Nov. 2013), 101–109.
- [KMJ*19] KOCH S., MATVEEV A., JIANG Z., WILLIAMS F., ARTEMOV A., BURNAEV E., ALEXA M., ZORIN D., PANOZZO D.: ABC: A big CAD model dataset for geometric deep learning. In *Proceedings of the 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (Long Beach, CA, US, 2019), pp. 9593–9603. <https://doi.org/10.1109/CVPR.2019.00983>.
- [LAK20] LOIZOU M., AVERKIOU M., KALOGERAKIS E.: Learning part boundaries from 3D point clouds. *Computer Graphics Forum* 39, 5 (Aug. 2020), 183–195.
- [LB16] LEE K. W., BO P.: Feature curve extraction from point clouds via developable strip intersection. *Journal of Computational Design and Engineering* 3, 2 (2016), 102–111.
- [LCBM21] LEJEMBLE T., COEURJOLLY D., BARTHE L., MELLADO N.: Stable and efficient differential estimators on oriented point clouds. *Computer Graphics Forum* 40, 5 (2021), 205–216.
- [MAM10] MAILLOT Y., ADAM B., MELKEMI M.: Shape reconstruction from unorganized set of points. In *Image Analysis and Recognition*. A. Campilho and M. Kamel (Eds.). Springer, Berlin, Heidelberg (2010), pp. 274–283. https://doi.org/10.1007/978-3-642-13772-3_28.
- [MOG11] MÉRIGOT Q., OVSIANIKOV M., GUIBAS L. J.: Voronoi-based curvature and feature estimation from point clouds. *IEEE Transactions on Visualization and Computer Graphics* 17, 6 (2011), 743–756.
- [MTAM*19] MOSCOSO THOMPSON E., ARVANITIS G., MOUSTAKAS K., HOANG-XUAN N., NGUYEN E., TRAN M., LEJEMBLE T., BARTHE L., MELLADO N., ROMANENGO C., BIASOTTI S., FALCIDIANO B.: Shrec'19 track: Feature curve extraction on triangle meshes. In *Proceedings of the 12th Eurographics Workshop 3D Object Retrieval 2019* (Gênes, Italy, May 2019), pp. 85–92. <https://doi.org/10.2312/3dor.20191066>.
- [MTB19] MOSCOSO THOMPSON E., BIASOTTI S.: Retrieving color patterns on surface meshes using edgeLBP descriptors. *Computers & Graphics* 79 (Apr. 2019), 46–57.
- [MTBDC20] MOSCOSO THOMPSON E., BIASOTTI S., DIGNE J., CHAINE R.: mpLBP: A point-based representation for surface pattern description. *Computers & Graphics* 86 (2020), 81–92.
- [NAK*18] NGUYEN K. W. L., A. A., KHAIRYANTO A., PANG W. C., SEET G. G. L., TOR S. B.: Edge detection from point cloud of worn parts. In *Proceedings of the 3rd International Conference on Progress in Additive Manufacturing (Pro-AM 2018)* (2018), 595–600. <https://doi.org/10.25341/D45C7S>.
- [NLNZ16] NI H., LIN X., NING X., ZHANG J.: Edge detection and feature line tracing in 3D-point clouds by analyzing geometric properties of neighborhoods. *Remote Sensing* 8, 9 (2016). <https://doi.org/10.3390/rs8090710>.
- [NS20] NOWAK M., SITNIK R.: High-detail animation of human body shape and pose from high-resolution 4D scans using iterative closest point and shape maps. *Applied Sciences* 10, 21 (2020). <https://doi.org/10.3390/app10217535>.
- [OPH96] OJALA T., PIETIKÄINEN M., HARWOOD D.: A comparative study of texture measures with classification based on featured distributions. *Pattern Recognition* 29, 1 (1996), 51–59.
- [Pie10] PIETIKÄINEN M.: Local binary patterns. *Scholarpedia* 5, 3 (2010), 9775.
- [PKG03] PAULY M., KEISER R., GROSS M.: Multi-scale feature extraction on point-sampled surfaces. *Computer Graphics Forum* 22, 3 (Sep. 2003), 281–289.
- [QSKG17] QI C. R., SU H., K. M., GUIBAS L. J.: PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (Hawaii, US, July 2017), pp. 77–85. <https://doi.org/10.1109/CVPR.2017.16>.
- [QSN*16] QI C. R., SU H., NIESSNER M., DAI A., YAN M., GUIBAS L. J.: Volumetric and multi-view CNNs for object classification on 3D data. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (Las Vegas, US, June 2016), pp. 5648–5656. <https://doi.org/10.1109/CVPR.2016.609>.

- [QYSG17] QI C. R., YI L., SU H. & GUIBAS L. J. [QYSG17], , , : PointNet++: Advances in neural information processing system, Long Beach, Los Angeles, US, pp. 5105–5114. <https://arxiv.org/abs/1706.02413>.
- [RBB09] RUSU R. B., BLODOW N., BEETZ M.: Fast point feature histograms (FPFH) for 3D registration. In *Proceedings of the 2009 IEEE International Conference on Robotics and Automation* (Kobe, Japan, May 2009), pp. 3212–3217. <https://doi.org/10.1109/ROBOT.2009.5152473>.
- [RC11] RUSU R. B., COUSINS S.: 3D is here: Point cloud library (PCL). In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (Shanghai, China, May 2011), IEEE, pp. 1–4. <https://doi.org/10.1109/ICRA.2011.5980567>.
- [SCZ21] SHU M., CHEN G., ZHANG Z.: 3D point cloud-based indoor mobile robot in 6-dof pose localization using a wi-fi-aided localization system. *IEEE Access* 9 (2021), 38636–38648.
- [SF04] STYLIANOU G., FARIN G.: Crest lines for surface segmentation and flattening. *IEEE Transactions on Visualization and Computer Graphics* 10, 5 (2004), 536–544.
- [SHWW20] SHENG Q., HONG R., WANG B., WANG Q.: Line matching of point clouds based on encoded asymmetric boundary. *IEEE Geoscience and Remote Sensing Letters* 17, 11 (Nov. 2020), 1983–1987.
- [TC98] TEICHMANN M., CAPPIS M.: Surface reconstruction with anisotropic density-scaled alpha shapes. In *VIS'98: Proceedings of the Visualization'98 (Cat. No.98CB36276)* (Research Triangle Park, NC, USA, 1998), pp. 67–72. <https://doi.org/10.1109/VISUAL.1998.745286>.
- [VTHLB15] VO A.-V., TRUONG-HONG L., LAEFER D. F., BERTOLOTTO M.: Octree-based region growing for point cloud segmentation. *ISPRS Journal of Photogrammetry and Remote Sensing* 104 (6 2015), 88–100.
- [WFDE13] WANG Y., FENG H., DELORME F., ENGIN S.: An adaptive normal estimation method for scanned point clouds with sharp features. *Computer-Aided Design* 45, 11 (11 2013), 1333–1348.
- [WHH10] WEBER C., HAHMANN S., HAGEN H.: Sharp feature detection in point clouds. In *Proceedings of the 2010 Shape Modeling International Conference* (Aix-en-Provence, France, June 2010), pp. 175–186. <https://doi.org/10.1109/SMI.2010.32>.
- [WXX*20] WANG X., XU Y., XU K., TAGLIASACCHI A., ZHOU B., MAHDAVI-AMIRI A., ZHANG H.: PIE-NET: Parametric inference of point cloud edges. In *Proceedings of the Advances in Neural Information Processing Systems* (Virtual-only Conference, 2020), H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan and H. Lin (Eds.), Curran Associates, Inc., vol. 33, pp. 20167–20178. <https://proceedings.neurips.cc/paper/2020/file/e94550c93cd70fe748e6982b3439ad3b-Paper.pdf>.
- [XH03] XU X., HARADA K.: Automatic surface reconstruction with alpha-shape method. *The Visual computer* 19, 7 (2003), 431–443.
- [XZW*15] XU J., ZHOU M., WU Z., SHUI W., ALI S.: Robust surface segmentation and edge feature lines extraction from fractured fragments of relics. *Journal of Computational Design & Engineering* 2, 2 (2015), 79–87.
- [YFFS20] YANG Y., FANG H., FANG Y., SHI S.: Three-dimensional point cloud data subtle feature extraction algorithm for laser scanning measurement of large-scale irregular surface in reverse engineering. *Measurement* 151 (2020), 107220.
- [YLT*20] YANG R., LI Q., TAN J., LI S., CHEN X.: Accurate road marking detection from noisy point clouds acquired by low-cost mobile lidar systems. *ISPRS International Journal of Geo-Information* 9, 10 (Oct. 2020), 608.
- [ZGW*16] ZHANG Y., GENG G., WEI X., ZHANG S., LI S.: A statistical approach for extraction of feature lines from point clouds. *Computers & Graphics* 56 (May 2016), 31–45.
- [Zho09] ZHONG Y.: Intrinsic shape signatures: A shape descriptor for 3D object recognition. In *Proceedings of the 2009 IEEE 12th International Conference on Computer Vision Workshops, ICCV Workshops* (Kyoto, Japan, 2009), pp. 689–696. <https://doi.org/10.1109/ICCVW.2009.5457637>.
- [ZWZ*18] ZOU Y., WANG X., ZHANG T., LIANG B., SONG J., LIU H.: BRoPH: An efficient and compact binary descriptor for 3D point clouds. *Pattern Recognition* 76 (Apr. 2018), 522–536.